Modules and User-Defined Modules

Introduction to Modules

What is a Module?

A **module** is a Python file (`.py`) that contains functions, classes, variables, and executable code. Instead of writing everything in a single file, we can divide a program into multiple modules. This makes the code easier to organize, reuse, and maintain.

Advantages of Modules

- Code Reusability
- Better Code Organization
- Reduces Code Duplication
- Easier Debugging
- Easier Maintenance
- Supports Team Collaboration

Types of Modules

Python provides three types of modules.

1. User-Defined Modules

Modules created by programmers.

Examples

- calculator.py
- student.py
- employee.py

2. Built-in Modules

Modules already provided by Python.

Examples

- math
- os
- random

- datetime
- statistics
- sys

Importing Modules

Python provides several ways to import modules.

Import Entire Module

Syntax

```python
import module_name
```

Example

```python
import my_module

print(my_module.greet("Rahul"))
```

Import Specific Function

Syntax

```python
from module_name import function_name
```

Example

```python
from my_module import greet

print(greet("Rahul"))
```

Import Multiple Functions

Syntax

```python
from module_name import function1, function2
```

Example

```python
from math import sqrt, factorial

print(sqrt(25))
print(factorial(5))
```

Import Using an Alias

Syntax

```python
import module_name as alias_name
```

Example

```python
import my_module as mm

print(mm.greet("Rahul"))
```

Import Everything (Not Recommended)

Syntax

```python
from module_name import *
```

Example

```python
from math import *

print(sqrt(16))
```

Note

Importing everything using * is not recommended because it may create name conflicts.

User-Defined Modules

Definition

A **User-Defined Module** is a Python file created by the programmer to store reusable code.

Step 1: Create a Module

Create a file named **my_module.py**

```python
def greet(name):
    return f"Hello, {name}!"

pi = 3.14159
```

Step 2: Import the Module

Create another file named **main.py**

```python
import my_module
print(my_module.greet("Alice"))
print(my_module.pi)
```

Output

```
Hello, Alice!
```

```
3.14159
```

Importing a Specific Function

```python
from my_module import greet

print(greet("Bob"))
```

Output

```
Hello, Bob!
```

Importing Using an Alias

```python
import my_module as m

print(m.greet("Charlie"))
```

Output

```
Hello, Charlie!
```

Python Module Search Path

Definition

When Python imports a module, it searches for that module in different locations.

Search Order

1. Current working directory
2. Built-in modules
3. Directories listed in `sys.path`

View the Search Path

```python
import sys
print(sys.path)
```

Add a Custom Path

```python
import sys
sys.path.append("C:/PythonModules")
```

Python will now also search this folder while importing modules.

The name Variable

Definition

Every Python file contains a special built-in variable called **`__name__`**. Python automatically assigns a value to this variable depending on how the file is executed.

There are only two possible values.

- `"__main__"`
- Module name

Case 1: Running the File Directly

Suppose we have a file named **demo.py**

```python
print(__name__)
```

Run the file.

```
None
python demo.py
```

Output

```
None
__main__
```

Explanation

Since **demo.py** is the file that starts the program, Python considers it the **main program**.

So Python automatically assigns

```
Python
__name__ = "__main__"
```

Case 2: Importing the File

Suppose **demo.py**

```
Python
print(__name__)
```

Another file **main.py**

```
Python
import demo
```

Output

```
None
demo
```

Explanation

This time, **demo.py** is not the main program. It is being used as a module. So Python assigns

```python
__name__ = "demo"
```

Here, `"demo"` is simply the filename without the `.py` extension.

if name == "main"

Definition

The statement

```python
if __name__ == "__main__":
```

checks whether the current file is being executed directly. If the file is executed directly, the condition becomes **True**. If the file is imported into another program, the condition becomes **False**.

Syntax

```python
if __name__ == "__main__":

    statements
```

Example

my_module.py

```python
def greet(name):

    return f"Hello {name}"

if __name__ == "__main__":

    print(greet("Alice"))
```

Run the File Directly

```
None
python my_module.py
```

Output

```
None
Hello Alice
```

Why?

When this file is executed directly,

```
Python
__name__ = "__main__"
```

Therefore,

```
Python
if __name__ == "__main__":
```

becomes

```
Python
if "__main__" == "__main__":
```

The condition is **True**, so the code inside the `if` block executes.

Import the Module

Create another file.

```
Python
import my_module
print("Welcome to Python")
```

Output

```
None
Welcome to Python
```

Why?

When **my_module.py** is imported, Python assigns

```
Python
__name__ = "my_module"
```

Now the condition becomes

```
Python
if "my_module" == "__main__":
```

The condition is **False**, so the code inside the `if` block is skipped. Only the imported functions, variables, and classes are available.

Why Do We Use if name == "main"?

- To prevent test code from running when the module is imported.
- To use the same file as both a standalone program and a reusable module.
- To improve code organization.
- To make modules reusable.

Installing External Modules

Third-party modules can be installed using **pip**.

Syntax

```
None
pip install module_name
```

Example

```
None
pip install requests
```